SAPIENZA
UNIVERSITÀ DI ROMA

Experimental Evaluation of a Multi-Layer Abstraction Approach for RL on Non-Markovian Tasks

Ingegneria Informatica, Automatica E Gestionale "ANTONIO RUBERTI"
Engineering in Computer Science

Matteo Ventali
ID number 1985026

Advisor
Prof. Fabio Patrizi

Academic Year 2025/2026

Dedicated to my family

Contents

Chapter 1

Introduction

Chapter 2

Background

2.1 Reinforcement Learning

Within the Machine Learning area, a general distinction is made between supervised, unsupervised, and Reinforcement Learning. Reinforcement Learning (RL) is a branch of Machine Learning (ML) concerned with learning how to make sequential decisions through interaction with an environment.
In *supervised learning*, an algorithm is trained on a labeled dataset containing examples, with the objective of learning a mapping that generalizes to samples not belonging to it. On the other hand, *unsupervised learning* works on unlabeled data and aims to identify underlying structures and patterns within the available observations [14].
Reinforcement Learning differs from both paradigms because the learner is not provided with explicit examples of the correct action to perform. Instead, an agent interacts with an (initially) unknown environment, selects actions, observes their consequences, and receives a reward signal that provides feedback about its behavior. Therefore, the goal of the agent is to learn a policy that maximizes the expected cumulative reward over time.
This type of interaction describes a *sequential decision-making problem*. Unlike prediction problems in which individual outputs can often be considered independently, in RL the consequences of a decision may extend beyond the immediate transition. An action selected at a given time may influence the state reached by the agent, the actions that will subsequently become available, and the rewards that can be obtained in the future. So, the quality of a decision cannot generally be evaluated only in terms of its immediate outcome, but must also account for its long-term consequences. In fact, in RL we often refer to trajectories (sequences of states and actions) generated by the agent's behavior during its interaction with the environment.
RL can be viewed as a framework for sequential decision-making under uncertainty, where an agent must learn how its actions influence both immediate and future outcomes. To reason formally about these interactions, a mathematical model of the decision process is required. The standard framework adopted for this purpose is the Markov Decision Process (MDP), which provides a formal description of states, actions, transition dynamics, and rewards. MDPs are introduced in the following

section and will be the basis for the RL methods discussed throughout this chapter.

2.1.1 Markov Decision Processes

The mathematical framework adopted for modeling sequential decision-making problems is the Markov Decision Process (MDP). An MPD is defined as a tuple

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle \tag{2.1}$$

where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, P is the transition probability function, R is the reward function, and $\gamma \in [0, 1]$ is the discount factor [14].
At each discrete time step t, the environment is in a state $S_t \in \mathcal{S}$. Based on this state, the agent selects an action $A_t \in \mathcal{A}$. The execution of the action triggers the environment to transition into a new state S_{t+1} and produces a scalar reward R_{t+1}.

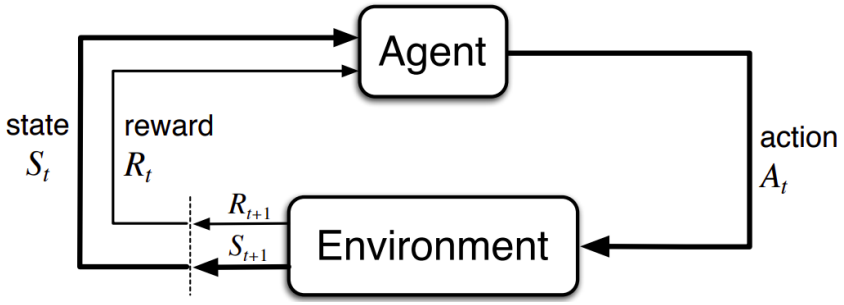

Figure 2.1. Interaction between the agent and the environment in an MDP [14].

The transition dynamics are characterized by

$$P(s' \mid s, a) = \Pr(S_{t+1} = s' \mid S_t = s, A_t = a) \tag{2.2}$$

which denotes the probability of reaching state s' after executing action a in state s. Looking at the P definition, it is immediate to notice that s' strictly depends only on the immediate previous state s. Therefore, with this definition, we are implicitly assuming the so-called *Markov property*: conditioned on the current state and action, the distribution of the next state is independent of the preceding interaction history. Formally, it can be expressed as follow:

$$\Pr(s' \mid s, a, S_{t-1}, A_{t-1}, \ldots, S_0) = \Pr(s' \mid s, a) \tag{2.3}$$

The current state must contain all the information relevant to predicting the future evolution of the process [14]. If additional information from the interaction history is required, the chosen state representation is *not Markovian* with respect to the problem under consideration.
The reward function assigns a numerical signal to the agent's interaction with the environment. The most general formulation corresponds to the function $R(s, a, s')$ which provides immediate feedback, whereas the agent's objective generally depends on rewards accumulated over multiple time steps. The distinction between immediate and long-term consequences is therefore central to sequential decision-making.

The discount factor γ determines the relative importance of future rewards. Values close to zero emphasize immediate outcomes, whereas values close to one assign greater importance to rewards received further in the future. The accumulation of discounted rewards will be formalized through the concept of return in Section 2.1.2. A sequence generated through interaction between the agent and the environment can be written as

$$s_0, a_0, r_1, s_1, a_1, r_2, s_2, \dots \tag{2.4}$$

and is referred to as a *trajectory*. The probability of a trajectory is determined jointly by the transition dynamics of the MDP and the actions chosen by the agent based on its own *policy*.

2.1.2 Policies, Returns, and Value Functions

A *policy* specifies how the agent selects actions in each state. A stochastic policy π assigns a probability to every action $a \in \mathcal{A}$ given a state $s \in \mathcal{S}$:

$$\pi(a \mid s) = \Pr(A_t = a \mid S_t = s) \tag{2.5}$$

For every state, these probabilities satisfy $\pi(a \mid s) \geq 0$ and $\sum_{a \in \mathcal{A}} \pi(a \mid s) = 1$. A deterministic policy is the special case in which one action is selected with probability one and can therefore be represented as a mapping $\pi : \mathcal{S} \to \mathcal{A}$.
The immediate reward obtained from a single transition is generally insufficient to evaluate an action, since its consequences may affect rewards received many steps later. The long-term outcome from time step t is represented by the *return*, defined as the discounted sum of future rewards:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \tag{2.6}$$

In episodic tasks, this sum terminates when a terminal state is reached. The discount factor γ controls the relative importance of immediate and delayed rewards. For continuing tasks with bounded rewards, choosing $\gamma < 1$ also ensures that the infinite sum remains finite.
Value functions measure the expected return obtained when the agent follows a given policy. The *state-value function* evaluates a state, whereas the *action-value function* evaluates the selection of a particular action in that state:

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s] \tag{2.7}$$

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a] \tag{2.8}$$

The expectation accounts for both the actions selected according to π and the stochastic transitions of the environment. The state-value and action-value functions are related by

$$V^\pi(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s)\, Q^\pi(s, a) \tag{2.9}$$

An optimal policy π^* maximizes the expected return from every state. The corresponding optimal value functions are

$$V^*(s) = \max_\pi V^\pi(s) \tag{2.10}$$

$$Q^*(s,a) = \max_\pi Q^\pi(s,a) \tag{2.11}$$

Once Q^* is known, an optimal deterministic policy can be obtained by selecting an action with maximum value:

$$\pi^*(s) \in \arg\max_{a \in \mathcal{A}} Q^*(s,a) \tag{2.12}$$

Policies and value functions therefore provide the principal quantities used to describe and compare decision-making strategies. Their recursive structure is captured by the Bellman equations [14].

2.1.3 Bellman Equations and Value Iteration

The return in Equation (2.6) can be decomposed into the immediate reward and the discounted return from the next time step:

$$G_t = R_{t+1} + \gamma G_{t+1} \tag{2.13}$$

This recursive relation leads to the *Bellman expectation equation*. For a policy π, the value of a state equals the expected immediate reward plus the discounted value of the successor state:

$$V^\pi(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s) \sum_{s' \in \mathcal{S}} P(s' \mid s,a) \left[R(s,a,s') + \gamma V^\pi(s') \right] \tag{2.14}$$

The corresponding equation for the action-value function is

$$Q^\pi(s,a) = \sum_{s' \in \mathcal{S}} P(s' \mid s,a) \left[R(s,a,s') + \gamma \sum_{a' \in \mathcal{A}} \pi(a' \mid s') Q^\pi(s',a') \right] \tag{2.15}$$

These equations express a consistency condition: the value assigned to the current state or action must agree with the values assigned to its possible successors. For an optimal policy, the expectation over the next action is replaced by a maximization, yielding the Bellman optimality equations

$$V^*(s) = \max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} P(s' \mid s,a) \left[R(s,a,s') + \gamma V^*(s') \right] \tag{2.16}$$

$$Q^*(s,a) = \sum_{s' \in \mathcal{S}} P(s' \mid s,a) \left[R(s,a,s') + \gamma \max_{a' \in \mathcal{A}} Q^*(s',a') \right] \tag{2.17}$$

When the transition and reward models are known, the optimal state-value function can be computed through *value iteration*. Starting from an arbitrary estimate V_0, the Bellman optimality backup is applied repeatedly:

$$V_{k+1}(s) = \max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} P(s' \mid s, a) \left[R(s, a, s') + \gamma V_k(s') \right] \tag{2.18}$$

For a finite discounted MDP, the sequence of estimates converges to V^*. An optimal policy can then be extracted by acting greedily with respect to the converged values:

$$\pi^*(s) \in \arg\max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} P(s' \mid s, a) \left[R(s, a, s') + \gamma V^*(s') \right] \tag{2.19}$$

Value iteration is a model-based planning method because it requires explicit knowledge of P and R. In RL, these quantities are unknown, motivating the adoption of model-free methods like Q-learning [14].

Policy Evaluation

Given a fixed policy π, the associated V^π can be computed by applying the Bellman expectation equation in an iterative way. Starting from a random initialization V_0, the update is

$$V^\pi_{k+1}(s) = \sum_{s' \in \mathcal{S}} P(s' \mid s, \pi(s)) \left[R(s, \pi(s), s') + \gamma V^\pi_k(s') \right] \tag{2.20}$$

for a deterministic policy π. The procedure is repeated until the value function converges. For finite discounted MDPs, iterative policy evaluation converges to V^π.

2.1.4 Value-Based Methods

Value-based RL methods compute an optimal policy by estimating value functions rather than representing the policy π directly. In particular, action-value methods learn the expected return associated with each state-action pair and derive a policy by selecting actions adopting a greedy strategy with respect to these estimates. These algorithms can be classified in two different categories:

- *on-policy*: the algorithm learns the values of the current policy. In other words, the data used for the policy update are generated adopting the same policy the algorithm is trying to improve.
- *off-policy*: the algorithm learns the values adopting a different policy with respect to the one is trying to improve. We usually refer to the latter as *Behavior Policy*, which controls directly the exploration and interaction with the environment during the learning phase.

Q-learning. Q-learning is a model-free and off-policy algorithm that estimates the optimal action-value function directly from observed transitions [16]. After observing the transition $(S_t, A_t, R_{t+1}, S_{t+1})$, it applies the temporal-difference update

$$Q_{t+1}(S_t, A_t) = Q_t(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_{a' \in \mathcal{A}} Q_t(S_{t+1}, a') - Q_t(S_t, A_t) \right] \tag{2.21}$$

where α is the learning rate hyper-parameter. The maximization defines a greedy target policy, while experience can be collected through an exploratory behavior policy such as ϵ-greedy. The most straightforward implementation of it is Tabular Q-learning which is particularly effective for small discrete MDPs, but it becomes impractical when the state space is large because it maintains a separate estimate for every (s, a) pair [14].

Deep Q-Networks. Deep Q-Networks (DQN) address the limitation of Q-learning by approximating the action-value function with a neural network $Q(s, a; \theta)$. To improve training stability, DQN stores transitions in a replay memory and learns from randomly sampled mini-batches. It also maintains a target network with parameters θ^-, which are periodically synchronized with the online parameters θ. For a transition (s, a, r, s', d), the target is

$$y^{\text{DQN}} = r + \gamma(1 - d) \max_{a' \in \mathcal{A}} Q(s', a'; \theta^-) \tag{2.22}$$

where d indicates whether the transition is terminal. The online network is trained by minimizing the squared difference between this target and $Q(s, a; \theta)$ [9].

Double Deep Q-Networks. The maximization in the DQN target can lead to overestimated action values because the same estimates are used both to select and evaluate the next action. Double Deep Q-Networks (DDQN) reduce this bias by using the online network for action selection and the target network for action evaluation:

$$y^{\text{DDQN}} = r + \gamma(1 - d) Q\left(s', \arg\max_{a' \in \mathcal{A}} Q(s', a'; \theta); \theta^-\right) \tag{2.23}$$

DDQN retains the replay memory, target network, and optimization procedure of DQN while changing only the construction of the temporal-difference target. This generally produces more reliable value estimates. [15].

2.2 Non-Markovianity and Temporally Extended Tasks

In several situations, a problem specification cannot be expressed as an objective that depends only on the last state of the environment. Specifically, in this scenarios, a successful behavior could depend on the order in which events occur, on whether certain conditions have already been satisfied, or on whether specific events are eventually reached during the execution.
For instance, consider a task requiring an agent to first visit a region (A) and subsequently reach a region (B). Reaching (B) cannot be considered sufficient to determine task completion unless we have not already reached region (A) in the past. Therefore, two trajectories ending in the region (B) may correspond to different levels of progress with respect to the task.
This types of objective specifications are commonly referred to as *temporally extended tasks*, since their satisfaction is defined over sequences of states or observations rather than on individual states only. When the information required to evaluate the task is

not completely represented by the current environment state, the task introduces a form of *non-Markovian* dependence on the interaction history (the *Markov property* is violated).
Representing such objectives requires a formalism capable of expressing temporal relationships between events. In this thesis, we choose to express these goal specifications trough Linear Temporal Logic on finite traces (LTL_f), a logical language which allows temporal properties to be specified over finite executions. Then, LTL_f specifications can in turn be converted in a deterministic finite automata (DFA), whose states provide a way to monitor the task progress.

2.2.1 Markovian and Non-Markovian Tasks

As introduced in Section 2.1.1, the Markov property requires the current state to contain all the information needed to characterize the future evolution of the environment [14].
In a MDP, the probability distribution $P(s' \mid s, a)$ of the next state depends only on the current state and action, and not on the complete interaction history.
A similar distinction can be made with respect to the task or reward structure.
In a *Markovian task*, the relevance of a transition can be determined from the current state, action, and successor state. So, under this assumption, a reward function can be expressed as $R(s, a, s')$.

However, there are some objectives that cannot be evaluated using only the current transition. In particular, their satisfaction depends on events that have occurred earlier in the execution. Let

$$h_t = s_0, a_0, s_1, a_1, \ldots, s_t \tag{2.24}$$

denote the interaction history up to time t. When the reward or the task satisfaction depend on h_t, we are referring to a *Non-Markovian task specification.*
This distinction is crucial because the environment itself may still satisfy the Markov property even when the task specification does not. In other words, non-Markovianity may arise from the specification of the objective rather than from the dynamics of the environment. The physical state can therefore be sufficient to predict the evolution of the environment while being insufficient to determine the progress made toward the task.

This concept is formalized by the *Non-Markovian Reward Decision Process* (NMRDP) definition introduced in [3].
An NMRDP is defined as a tuple:

$$\mathcal{M}_{\text{NM}} = \left\langle \mathcal{S}, \mathcal{A}, P, \overline{R}, \gamma \right\rangle \tag{2.25}$$

where $\mathcal{S}$, $\mathcal{A}$, P, and γ have the same meaning as in an MDP, while the reward function is defined over finite sequences of state-action pairs:

$$\overline{R} : (\mathcal{S} \times \mathcal{A})^* \to \mathbb{R} \tag{2.26}$$

Let

$$\tau = \langle (s_0, a_0), (s_1, a_1), \ldots, (s_{n-1}, a_{n-1}) \rangle \tag{2.27}$$

be a finite trace, and let $\tau_{\leq i}$ denote its prefix containing the first i state-action pairs. The value of τ is defined as

$$V(\tau) = \sum_{i=1}^{|\tau|} \gamma^{i-1} \overline{R}(\tau_{\leq i}) \tag{2.28}$$

Unlike a Markovian policy, whose action depends only on the current state, a policy for an NMRDP may depend on the complete sequence of states observed so far:

$$\overline{\pi} : \mathcal{S}^* \to \mathcal{A} \tag{2.29}$$

Therefore, both the reward and the policy may depend on the interaction history rather than exclusively on the current state.

2.2.2 Linear Temporal Logic on Finite Traces

Linear Temporal Logic (LTL) [12] is a formal language used to describe properties of sequences that evolve over time. Unlike propositional logic, which evaluates formulas with respect to a single state (the *interpretation*), temporal logic allows the specification of relationships between events occurring at different points of an execution.
LTL is interpreted over infinite traces. However, in episodic RL, interactions agent-environment are naturally represented by finite sequences, since an episode terminates after a finite number of steps. Therefore, in this thesis, we mainly consider LTL_f which is a variant of LTL evaluated on finite traces [7].

Let P be a finite set of atomic propositions describing relevant properties of the environment. At each time step, a subset of these propositions is true. A finite trace over P can therefore be represented as

$$\rho = \rho_0, \rho_1, \ldots, \rho_{n-1} \tag{2.30}$$

where

$$\rho_i \in 2^P \tag{2.31}$$

denotes the set of atomic propositions that hold at position i.
An LTL_f formula is built from atomic propositions using boolean and temporal operators. Formally,

$$\varphi ::= A \mid (\neg\varphi) \mid (\varphi_1 \wedge \varphi_2) \mid (\circ\varphi) \mid (\varphi_1 \, U \, \varphi_2) \mid (\Diamond\varphi) \mid (\Box\varphi) \tag{2.32}$$

where $A \in P$, $\circ$ is the *next* operator, U is the *until* operator, $\Diamond$ is the *eventually* operator and $\Box$ it the *globally* operator.
The satisfaction of an LTL_f formula is defined with respect to a position of a finite trace. For an atomic proposition p,

$$\rho, i \models p \quad \iff \quad p \in \rho_i \tag{2.33}$$

A finite trace ρ satisfies an LTL_f formula φ, written as

$$\rho \models \varphi \tag{2.34}$$

when the formula is satisfied at the initial position of the trace. Hence, every LTL_f formula defines a set of finite traces that satisfy the corresponding temporal specification.

2.2.3 Deterministic Finite Automata

A Deterministic Finite Automaton (DFA) [2] is a finite-state machine useful to recognize languages defined over a finite alphabet. Formally, a DFA is defined as a tuple

$$\mathcal{A} = \langle Q, \Sigma, \delta, q_0, F \rangle \tag{2.35}$$

where Q is a finite set of states, Σ is a finite input alphabet, δ is the transition function

$$\delta : Q \times \Sigma \to Q \tag{2.36}$$

$q_0 \in Q$ is the initial state, and $F \subseteq Q$ is the set of accepting states.
The automaton processes an input word one symbol at a time. Since the transition function is deterministic, for every state $q \in Q$ and input symbol $\sigma \in \Sigma$, there exists a unique successor state

$$q' = \delta(q, \sigma) \tag{2.37}$$

Given a finite word

$$w = \sigma_0 \sigma_1 \dots \sigma_{n-1} \in \Sigma^* \tag{2.38}$$

the execution, or run, of the automaton over w is the sequence of states

$$q_0, q_1, \dots, q_n \tag{2.39}$$

such that

$$q_{i+1} = \delta(q_i, \sigma_i) \qquad 0 \leq i < n \tag{2.40}$$

The word w is accepted by the automaton if the state reached after processing the complete word belongs to the accepting set:

$$q_n \in F \tag{2.41}$$

The set of all finite words accepted by $\mathcal{A}$ defines the language recognized by the automaton, and it is denoted by

$$L(\mathcal{A}) = \{w \in \Sigma^* \mid \mathcal{A} \text{ accepts } w\} \tag{2.42}$$

2.2.4 From LTL$_f$ Specifications to DFA

A LTL$_f$ formula can be represented trough an equivalent DFA. As described in [7], this property is achieved thanks to the fact that every LTL$_f$ formula defines a regular language over finite traces.
Formally, let's consider the LTL$_f$ formula φ defined over a set P of atomic propositions. We can define the DFA $\mathcal{A}_\varphi = \langle Q, 2^P, \delta, q_0, F \rangle$ in such a way that for every finite trace ρ holds:

$$\rho \models \varphi \quad \iff \quad \rho \in L(\mathcal{A}_\varphi) \tag{2.43}$$

Therefore, the LTL$_f$ formula and the corresponding DFA provide two different representations of the same temporal specification. In particular, the former provides a declaratively way to describe a temporal specification, instead the latter provides an operational representation useful for the integration of the formula in an algorithm.

The DFA state summarizes the information contained in the observed trace that is relevant to the satisfaction of the temporal specification. Hence, we can interpret it as a finite-memory representation of the progress made toward the task.
This representation is particularly useful for temporally extended tasks. If we consider different trajectories that lead the environment in the same physical state at a certain time, we will be able to identify the current correct stage of the temporal objective in each trajectory. Hence, the environment state and the automaton state can be considered jointly through an augmented representation

$$\tilde{s}_t = (s_t, q_t), \tag{2.44}$$

where s_t represents the physical configuration of the environment and q_t corresponds to the logical progress with respect to the LTL$_f$ specification. This representation makes the task-relevant history explicitly available to the learning process and it is the basis for the treatment of temporally extended objectives adopted in the framework presented in Chapter 3.

This construction has been formalized in the literature on NMRDP. In particular, in [4] it is shown that LTL$_f$/LDLf non-Markovian rewards can be compiled into an equivalent Markovian MDP by augmenting the environment state with the states of the corresponding automata.

2.3 Reward Design in Reinforcement Learning

The reward design task plays a central role in a RL setting. As depicted in Figure 2.1, the reward received by the agent represents a feedback related on its actions. Therefore, the reward function can be considered as an encoding of which actions are desirable with respect to the task objective required by the problem. Unfortunately, it doesn't exist a general method to the define a reward function. Each reward function must be suitable and informative for the problem under consideration. In a common scenario, the most intuitive reward function corresponds to feeding the agent with a reward only at the end of the task. Although this formulation is trivial,

it makes the learning process more difficult. On the other hand, defining a reward function that generates also intermediate feedback could be cumbersome and not straightforward.

In this section, we report some notions about reward functions with a particular focus to sparse reward settings. Then, we describe the Reward Shaping mechanism useful to enrich in some way the reward signal that comes out from the reward function.

2.3.1 Goal-Oriented Tasks and Sparse Rewards

Many RL problems are naturally formulated in terms of achieving a specific objective. In these scenarios, the agent receives a reward signal only when it reaches the final goal, and therefore it is not guided through a reward signal that continuously assesses every intermediate action. Such problems can be modeled as goal-oriented MDPs, in which the task objective is explicitly associated with a goal condition.

Let g denote a goal and let $\mathcal{G}_g \subseteq \mathcal{S}$ be the set of states that satisfy it. A simple sparse formulation assigns a positive reward only when a transition reaches a goal state:

$$R_g(s, a, s') = \begin{cases} r_g, & \text{if } s' \in \mathcal{G}_g, \\ 0, & \text{otherwise} \end{cases} \tag{2.45}$$

where $r_g > 0$ is the reward associated with successful goal achievement.
Depending on the setting, reaching a goal state may also terminate the episode.

A reward function is described as *sparse* when the agent receives informative feedback only on a small fraction of the transitions it experiences. On the other hand, when the agent receives a high frequency reward signal from the environment, we refer to *dense* reward function.

A notable sparse-reward setting is the *goal MDP*: an MDP $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$ is a *goal MDP* if and only if there exists a set $\mathcal{G} \subseteq \mathcal{S}$ of goal states for which the reward function satisfies

$$R(s, a, s') = \begin{cases} 1, & \text{if } s \notin \mathcal{G} \text{ and } s' \in \mathcal{G}, \\ 0, & \text{otherwise} \end{cases} \tag{2.46}$$

In addition, goal states have zero optimal continuation value:

$$V^*(s) = 0, \qquad \forall s \in \mathcal{G} \tag{2.47}$$

Under these conditions, the agent receives a unit reward only when it enters the goal set from a non-goal state. Once a goal state has been reached, no further return can be accumulated from that state.
Sparse rewards provide a direct representation of the task objective because they reward behavior that actually achieves the final goal. However, they also make learning more difficult [14, 6]. Before encountering a successful trajectory, the agent

may receive the same reward for many different actions and states, with no direct indication of whether its behavior is moving toward or away from the objective. This difficulty is closely related to exploration. Without informative intermediate feedback, the agent must discover successful behavior largely through interaction with the environment. If the goal can be reached only after a long sequence of appropriate decisions, the probability of finding a successful trajectory without the reward signal's guidance may be very low.
Sparse-reward problems will be central into the framework described in Chapter 3.

2.3.2 Reward Shaping and PBRS

To address the consequences of sparse reward settings, in the literature has been introduced a technique called Reward Shaping. The core idea behind reward shaping is to augment the reward signal generated by the reward function of the original MDP with an additional signal [10]. This additional signal provides intermediate feedback to the agent during the learning process.

Formally, given a MDP $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$ a shaping function F is useful to define a modified reward function:

$$R'(s, a, s') = R(s, a, s') + F(s, a, s') \tag{2.48}$$

The introduction of F transforms the original MDP M to a modified version $\mathcal{M}' = \langle \mathcal{S}, \mathcal{A}, P, R', \gamma \rangle$. This transformation could modify the desirability of trajectories and the optimal policy induced by the original reward function.

Potential Based Reward Shaping

One of the most adopted solution in the designing of the shaping function F is the Potential Based Reward Shaping (PBRS) [10]. Formally,

$$F(s, a, s') = \gamma \Phi(s') - \Phi(s) \tag{2.49}$$

From the definition, it is immediate to see that the function $F(s', a, s)$ does not depend on the action a and therefore it could be defined simply as $F(s, s')$. Moreover, there is the introduction of $\Phi(s)$ which is the *Potential Function*. Specifically,

$$\Phi : S \to \mathbb{R} \tag{2.50}$$

assigns a scalar potential value to each state. Despite the difference in meaning between Φ and V-function, they share a structural similarity in their definition. In fact, the V-function is a practical and common choice when the reward shaping technique is adopted.

Policy Invariance of PBRS

The introduction of the shaping term according on PBRS formulation does not alter the optimal policies of the original MDP, if we assume the following facts:

- the discount factor $\gamma \in [0, 1)$;

- the potential function Φ is bounded.

This result relies on the fact that the shaping contribution accumulated along a trajectory assumes a telescoping structure. Considering a trajectory

$$s_0, s_1, \ldots, s_T$$

the discounted cumulative shaping reward is

$$\sum_{t=0}^{T-1} \gamma^t \left(\gamma \Phi(s_{t+1}) - \Phi(s_t)\right) = -\Phi(s_0) + \gamma^T \Phi(s_T) \tag{2.51}$$

Under the assumptions:

$$\lim_{T \to \infty} \gamma^T \Phi(s_T) = 0$$

and therefore

$$\sum_{t=0}^{\infty} \gamma^t F(s_t, s_{t+1}) = -\Phi(s_0) \tag{2.52}$$

For a fixed initial state s_0, the shaped return differs from the original one only for $-\Phi(s_0)$. Hence, the relative ordering of policies is preserved, and any optimal policy in the original MDP remains optimal in the shaped MDP [10].
When we are in episodic finite-horizon settings, additional care is required because the terminal term $\gamma^T \Phi(s_T) \neq 0$. A common sufficient condition is to assign zero potential to terminal states:

$$\Phi(s) = 0, \qquad \forall s \in \mathcal{S}_{\text{terminal}} \tag{2.53}$$

so that, the cumulative shaping contribution again reduces to $-\Phi(s_0)$. Under this condition, the shaping transformation preserves the original task objective also in the episodic case.

Chapter 3

Multi-Layer Abstraction Framework

The foundations of this thesis lie in the multi-level framework introduced in [6]. The framework exploits abstract representations of the original decision process in order to tackle the difficulties associated with a sparse-reward learning problem.

In this chapter, we firstly analyze the main components of the reference framework including the abstraction hierarchy and the learning architecture adopted. Then, we move on presenting the resulting end-to-end algorithm. In the end, we describe its implementation and the integration of non-Markovian goal specifications trough LTL_f.

3.1 State Abstraction and Abstract Guidance

The core idea behind the framework corresponds to the linear hierarchy of MDPs built over the ground domain. In particular, each level of the stack can be seen as a simplification of the one immediately below. The purpose of this hierarchical organization is to exploit simpler representation of the original decision problem in order to derive additional information that can guide the learning process in a more efficient way.
Let

$$\mathcal{M}_0, \mathcal{M}_1, \dots, \mathcal{M}_L$$

denote the sequence of MDPs forming the hierarchy, where $\mathcal{M}_0$ represents the ground MDP. Therefore, the hierarchy can be conceptually represented as

$$\mathcal{M}_L \to \mathcal{M}_{L-1} \to \cdots \to \mathcal{M}_0$$

where the information flows according to the arrows.

3.1.1 State Abstraction mechanism

Each component of the hierarchy is a complete MDP $\mathcal{M}_i = \langle \mathcal{S}_i, \mathcal{A}_i, P_i, R_i, \gamma_i \rangle$. Given two consecutive levels of the hierarchy, $\mathcal{M}_i$ and $\mathcal{M}_{i+1}$, the latter represents a

simplified model of the former. They are connected trough an abstraction mapping, which is a function that maps the states of $\mathcal{M}_i$ into the states of $\mathcal{M}_{i+1}$. Formally the mapping function is

$$\varphi_i : \mathcal{S}_i \to \mathcal{S}_{i+1} \tag{3.1}$$

and the pair $\langle \mathcal{M}_{i+1}, \varphi_i \rangle$ is the abstraction built over $\mathcal{M}_i$. Unlike other HRL frameworks, this one does not impose any constraint about the mapping between $\mathcal{A}_i$ and $\mathcal{A}_{i+1}$. This feature leaves freedom and flexibility to the designer during the definition of the abstractions. Therefore, each MDP can be equipped with an appropriate set of actions suitable for each level of the problem description. The only assumptions made in [6] are:

- $\mathcal{M}_0$ is a Goal MDP;
- if $\mathcal{M}_i$ is a goal MDP with goal states $\mathcal{G}_i$ and $\langle \mathcal{M}_{i+1}, \varphi_i \rangle$ is the abstraction built over it, then $\mathcal{M}_{i+1}$ is a goal MDP such that:

$$\mathcal{G}_{i+1} = \bigcup_{s \in \mathcal{G}_i} \varphi_i(s) \tag{3.2}$$

3.1.2 Guidance from Abstract Models

Once the abstraction hierarchy has been defined, the framework exploits the higher-level models to derive guidance for the learning process at lower levels. The main idea is to solve an abstract MDP and use the resulting information in some way to speed-up the learning phase at the lower level.

Let $\mathcal{M}_{i+1}$ an MDP of the hierarchy. We can solve it using planning or learning techniques. In particular, we can adopt the first solutions when the MDP is fully-known, including the transition function and the reward function. This is not always true, especially when we are considering a rich and complex abstraction over the real problem. For instance, an abstraction that includes in its dynamics physics relations and quantities.
In the framework, the information extracted at level $(i+1)$ corresponds to V_{i+1}. Then, this term is plugged inside the PBRS formulation described in Section 2.3.2:

$$F_i(s, a, s') = \gamma V_{i+1}(s') - V_{i+1}(s) \tag{3.3}$$

$F_i(s, a, s')$ corresponds to the shaping term exploited to enrich the reward function at level i. As a consequence of this choice, transitions in $\mathcal{M}_i$ that leads towards states mapped to abstract ones with higher potential values receive an additional positive feedback in the reward signal. On the other hand, transitions towards states associated with lower potential in the abstraction can receive a negative shaping contribution.

3.2 Reward Shaping and Learning Architecture

Although the framework adopts a potential-based shaping formulation, its shaping mechanism differs from the standard return-invariant setting.

Episodic tasks are common settings to be faced in RL. In this situations, preserving the return-invariance property of PBRS requires a particular treatment of terminal states, as discussed in Section 2.3.2. While this property guarantees that the shaped and original MDPs share the same optimal policies, it may also limit the ability of the shaping signal to provide persistent exploration guidance.

3.2.1 Non Return-Invariant Reward Shaping

The return-invariance property of PBRS can be preserved by assigning a null potential to terminal states. Under this condition, for a trajectory $s_0, s_1, \ldots, s_T$, the cumulative discounted shaping contribution is

$$G = -V(s_0) + \gamma^T V(s_T) \tag{3.4}$$

and since $V(s_T) = 0$, the previous expression reduces to $-V(s_0)$.

It is immediate to notice that, for trajectories originating from the same initial state (s_0 is the same) the cumulative shaping reward is independent of the intermediate states visited. Hence, in a sparse reward problem like a goal MDP, all finite trajectories that terminate without reaching the objective will receive the same shaped return contribution, regardless of whether they end close to or far from the desired goal.

To overcome this limitation, the authors of [6] introduce a modified version of PBRS called *non return-invariant Reward Shaping*. In practice, the constraint about the final state value is relaxed.
Now, two unsuccessful trajectories ρ_1 and ρ_2, originating from the same state, can satisfy

$$G(\rho_1) > G(\rho_2)$$

retaining useful information about the quality of the trajectories, with respect to the distance between the final state and the final goal. In other words, adopting this version, the shaping mechanism provides a persistent bias toward states associated with higher abstract values.

3.2.2 Biased and Unbiased Learning

The non return-invariant shaping mechanism loses the guarantee that the optimal policy of the shaped MDP corresponds to the optimal one in the original MDP.
From this consideration, it's now important to consider two MDPs at each level in the hierarchy. Precisely, we have:

- $\mathcal{M}^u = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$ named *unbiased* MDP. The reward function R is not influenced by the shaping signal;
- $\mathcal{M}^b = \langle \mathcal{S}, \mathcal{A}, P, R', \gamma \rangle$ named *biased* MDP. Here, the reward function R' involves also the shaping signal according to the non return-invariant shaping formulation.

At this point, the task is to learn an optimal policy in $\mathcal{M}^u$ exploiting the information coming from the learning process performed on $\mathcal{M}^b$. Hence, from now on, we have two learning procedures over the two different MDPs.

The learning procedures are coupled together in such a way we can exploit the advantage coming from the shaping signal without losing the guarantee of reaching the optimal policy in the unbiased MDP. In particular, the biased learner is responsible for interacting with the environment and therefore it establishes the behavior policy used during data collection. The key idea supporting this architecture is to exploit the trajectories generated by the biased learner in $\mathcal{M}^b$ to update also the unbiased one.
Formally, for each transition generated during the interaction with the environment

$$(s_t, a_t, r_{t+1}, s_{t+1})$$

both learning processes are updated using the same experience, but with different reward signals. The biased learner uses

$$r'_{t+1} = r_{t+1} + F(s_t, a, s_{t+1})$$

while the unbiased learner uses only the original reward r_{t+1}. In this way, the unbiased learner is able to see trajectories toward the final goal allowing it to learn the optimal desired policy.
It is worth to mention that this architecture requires the adoption of an off-policy algorithm to implement the unbiased learner. This concept is obvious considering that transitions comes out from a different algorithm (the biased learner one).

3.3 End-to-End Framework

The overall procedure described in the previous sections is summarized in the following algorithm extracted and adapted from [6].

Algorithm 1: Full learning architecture

Input: An off-policy learning algorithm $\mathcal{L}$
Input: The hierarchy $\mathcal{M}_0, \dots, \mathcal{M}_L$ and the abstraction mappings $\varphi_0, \dots, \varphi_{L-1}$
Output: An estimate $\widehat{\pi}^*_0$ of the optimal policy for the ground MDP

$\widehat{V}^*_{L+1}(s) \leftarrow 0$;
$\varphi_L(s) \leftarrow s$;
for $i \leftarrow L, L-1, \dots, 0$ **do**
  $F_i \leftarrow \text{Shaping}\big(\gamma_i, \varphi_i, \widehat{V}^*_{i+1}\big)$;
  $\text{Learner}^u_i \leftarrow \mathcal{L}(\mathcal{M}_i)$;
  $\text{Learner}^b_i \leftarrow \mathcal{L}(\mathcal{M}_i)$;
  while $\neg\, \text{Learner}^b_i.\text{Stop}()$ **do**
    $s \leftarrow \mathcal{M}_i.\text{State}()$;
    $a \leftarrow \text{Learner}^b_i.\text{Action}(s)$;
    $(r, s') \leftarrow \mathcal{M}_i.\text{Act}(a)$;
    $r^b \leftarrow r + F_i(s, a, s')$;
    $\text{Learner}^b_i.\text{Update}(s, a, r^b, s')$;
    $\text{Learner}^u_i.\text{Update}(s, a, r, s')$;
  $\widehat{\pi}^*_i \leftarrow \text{Learner}^u_i.\text{Output}()$;
  $\widehat{V}^*_i \leftarrow \text{ComputeValue}(\widehat{\pi}^*_i, \mathcal{M}_i)$;

The algorithm highlights two important features of the learning architecture:

1. after completing the learning process at level i, the value function used to construct the shaping signal for the level immediately below is derived from the unbiased learner. In particular, the policy produced by the unbiased learner is used to compute the corresponding value function V_i^*, which then serves as the source of guidance for level $i-1$;

2. the final result of the procedure is the policy $\widehat{\pi}_0^*$ learned by the unbiased learner at the ground level.

Hence, it is clear that the biased policies are used only as intermediate mechanisms to improve the exploration process for the unbiased learner.

3.4 Implementation and Design Choices

The framework described in the previous sections represents the foundation of this thesis. Starting from the architecture and the algorithm introduced in [6], a full implementation of the multi-level framework has been developed with the aim to experimentally evaluate its effectiveness against temporally extended non-Markovian task specifications in a complex domain.
Until now, the original framework has been evaluated on relative simple tasks and domains. Moreover, when a temporally extended behavior was required, there was a need to manually define the corresponding DFA for the task.
In this thesis, we address this limitation introducing explicitly the support for task specifications expressed in LTL_f. Furthermore, the whole training pipeline has been fully-automated in such a way to be transparent and *ready-to-use* for the user.

3.4.1 Framework Architecture

The implementation developed in this thesis follows the multi-level architecture described in the previous sections, while extending it with an explicit representation of temporally extended tasks. The temporal objective is specified trough an LTL_f formula, which is automatically translated into a DFA taking advantage of the *ltlf2dfa* Python library. The state q of the automaton is shared across each level in the hierarchy, so that all abstraction levels are synchronized on the same temporal objective over different representations of the environment.

The hierarchy is processed from the highest abstraction level toward the ground MDP. The highest-level MDP is solved without receiving shaping guidance. When its transition and reward models are explicitly available and the state space is sufficiently small, we can also adopt a Value Iteration algorithm to solve the abstract task. Once a level has been solved, the resulting V-function is propagated to the level immediately below and used to construct its shaping signal according to the mechanism described in Algorithm 1. This procedure is repeated throughout the hierarchy until the ground level is reached.

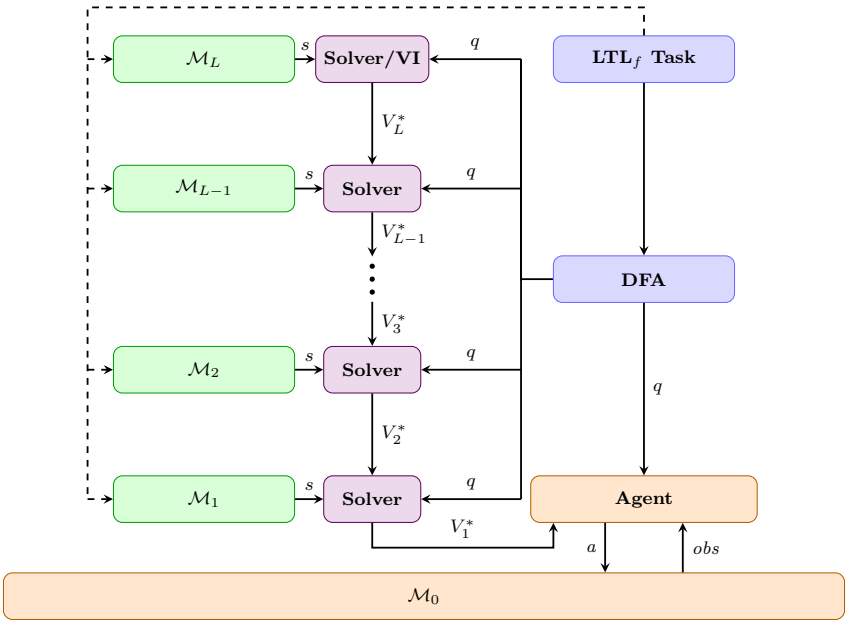

Figure 3.1. Full framework architecture.

At each level, the V-function propagated to the level immediately below is derived from the solution produced by the unbiased learner. The framework supports two different ways to extract the unbiased V-function.
The first possibility consists to extract the greedy policy induced by the unbiased Q-function following

$$\pi^u(s) = \arg\max_{a \in \mathcal{A}} Q^u(s, a) \tag{3.5}$$

The, V^{π^u} is computed trough iterative Policy Evaluation, as introduced in Section 2.1.3. When this solution is adopted, the numerical magnitudes and errors of Q^u are not propagated directly.
Alternatively, the V-function can be obtained by estimating it following

$$\hat{V}^u(s) = \max_{a \in \mathcal{A}} Q^u(s, a) \tag{3.6}$$

Therefore, the magnitudes and estimation errors are propagated trough the shaping mechanism. A comparative study of the two solution will be presented in Chapter 4.

3.4.2 Temporal Task Specification

The implementation supports two different classes of temporally extended tasks:

- episodic tasks: they are expressed trough a LTL$_f$ formula;
- cyclic tasks: they are expressed trough a dedicated automaton whose transitions encode the phases of the recurring task.

Episodic tasks

For episodic tasks, reaching an accepting state of the DFA corresponds to the completion of the temporal objective. The associated goal reward is generated and the current interaction episode terminates. At the beginning of a new episode, both the environment and the automaton state are reinitialized.

Cyclic tasks

For cyclic tasks, reaching the end of one temporal sequence does not terminate the overall objective. Instead, the task progress is redirected to the initial phase of the cycle, allowing the same temporal behavior to be executed again during the same episode. Hence, a LTL_f is not suitable to express this type of required interaction. However, the corresponding automaton is designed to contain transitions that explicitly represent this recurrence.

3.4.3 Configuration Interface and Automated Pipeline

The complete framework is configurable through two external JSON files, one describing the abstraction hierarchy and one specifying the temporal task. This design separates the experimental configuration from the implementation of the learning procedure, allowing different tasks and abstraction settings to be tested without modifying the underlying source code.

The abstraction hierarchy is defined through an `abstraction.json` file. This configuration specifies the number of abstraction levels and the parameters required to instantiate and solve the corresponding abstract MDPs. A possible configuration is the following one:

```
{
    "levels": [
        {
            "name": "level1", "grid_w": 360, "grid_h": 360,
            "learning": {
                "episodes": 1000000,
                "max_steps": 600,
                "alpha": 0.1,
                "epsilon_start": 1.0,
                "epsilon_min": 0.05,
                "epsilon_decay": 0.999995,
                "gamma_shaping": 1.0
            }
        },
        {
            "name": "level2", "grid_w": 180, "grid_h": 180,
            "algorithm": "value_iteration"
        }
    ]
}
```

Listing 3.1. Example of an abstraction hierarchy configuration.

The temporal task is specified through a separate configuration file `trajectory.json`. In the episodic case, the configuration contains the LTL_f formula together with the definitions required to evaluate the atomic propositions appearing in it.
In the cyclic case, the recurring temporal structure is explicitly encoded and from that specification the automaton is automatically built.

A possible episodic task configuration is the following one:

```
{
    "formula": "F(wp1 & X(F(low & g1)))",
    "goal_reward": 10000,
    "regions": {
        "wp1": {
            "center": [-0.5, 1.0],
            "radius": 0.1
        },
        "g1": {
            "center": [0.5, 0.5],
            "radius": 0.1
        }
    },
    "predicates": {
        "low": {
            "type": "half_plane",
            "axis": "y",
            "operator": "<",
            "threshold": 0.7
        }
    }
}
```

Listing 3.2. Example of an LTL_f task configuration.

A possible cyclic task configuration is the following one:

```
{
    "task_type": "cyclic_waypoints",
    "goal_reward": 10000,
    "waypoint_cycle": ["g1", "g2", "g3", "g4"],
    "regions": {
        "g1": {"center": [-0.75, 1.0625], "radius": 0.12},
        "g2": {"center": [0.583333, 1.0625], "radius": 0.12},
        "g3": {"center": [0.583333, 0.4375], "radius": 0.12},
        "g4": {"center": [-0.75, 0.4375], "radius": 0.12}
    }
}
```

Listing 3.3. Example of a cyclic task configuration.

Chapter 4

Experimental Evaluation

In this chapter, we present the experimental evaluation of the implemented framework applied to complex domains with temporally extended non-Markovian task specifications.
The experiments have been designed to investigate the effectiveness and applicability of the learning architecture under different task and abstraction configurations.
Throughout this chapter, we will firstly evaluate the framework on episodic tasks of increasing temporal complexity, then we will study the effect of introducing multiple abstraction levels, and finally we will consider its application on cyclic temporal objectives.

4.1 Evaluation Goals and Research Questions

The target of this experimental analysis is to identify possible limitations of the framework and to better understand its potential when increasingly complex tasks must be fulfilled. The experimental evaluation pipeline is organized around four key research questions, each one investigating a different aspect of the framework.

Our research question are:

- **RQ1** To what extent can the theoretical framework be directly applied to a complex domain without requiring additional adaptations?
- **RQ2** How does the multi-level framework perform on episodic non-Markovian tasks with a growing temporal complexity?
- **RQ3** How does the introduction of multiple abstraction levels affect the guidance signal and the resulting learning process?
- **RQ4** Can the framework be applied to learn cyclic temporal behaviors that are not limited to purely episodic task structures?

These research questions guide the experimental analysis presented in the following sections, covering the practical limitations of the learning architecture, the effect of temporal-task complexity, the role of multi-level abstraction, and the extension to cyclic task structures.

4.2 Experimental Methodology

4.2.1 Experimental Environments

4.2.2 Temporal Tasks and Predicates

4.2.3 Abstraction Configurations

4.2.4 Learning Setup

4.2.5 Training and Evaluation Protocol

4.2.6 Evaluation Metrics

4.3 RQ1: Practical Limitations of Ground-Level Guidance Transfer

4.3.1 Biased–Unbiased Learning

4.3.2 Temporal Alignment of the Shaping Signal

4.3.3 Shaping Signal Quality Analysis

4.3.4 Discussion

4.4 RQ2: Performance under Increasing Temporal Complexity

4.4.1 Experimental Design

4.4.2 Learning Performance

4.4.3 Greedy Evaluation

4.4.4 Final Policy Evaluation

4.4.5 Discussion

4.5 RQ3: Effect of Multi-Level Abstraction

4.5.1 Experimental Design

4.5.2 Learning Performance

4.5.3 Analysis of the Guidance Signal

4.5.4 Effect of Task Complexity on Multi-Level Guidance

4.5.5 Discussion

4.6 RQ4: Effectiveness on Cyclic Temporal Tasks

4.6.1 Task Definition and Cyclic Automaton

4.6.2 Experimental Design

4.6.3 Learning Behaviour and Results

4.6.4 Discussion

4.7 Additional Case Studies

4.8 Overall Discussion

Chapter 5

Conclusions and Future Work

5.1 Summary of the Approach

5.2 Main Findings

5.3 Contributions

5.4 Limitations

5.5 Future Research Directions

5.6 Concluding Remarks

Bibliography

[1] Abel, D., Umbanhowar, N., Khetarpal, K., Arumugam, D., Precup, D., and Littman, M. Value preserving state-action abstractions. In *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics* (edited by S. Chiappa and R. Calandra), vol. 108 of *Proceedings of Machine Learning Research*, pp. 1639–1650. PMLR (2020). Available from: https://proceedings.mlr.press/v108/abel20a.html.

[2] Ausiello, G., D'Amore, F., and Gambosi, G. *Linguaggi, modelli, complessità*. Franco Angeli (2003). ISBN 9788846444707.

[3] Bacchus, F., Boutilier, C., and Grove, A. Rewarding behaviors. AAAI'96, p. 1160–1167. AAAI Press (1996).

[4] Brafman, R. I., Giacomo, G. D., and Patrizi, F. Ltlf/ldlf non-markovian rewards. In *AAAI Conference on Artificial Intelligence* (2018).

[5] Canonaco, G., Ardon, L., Pozanco, A., and Borrajo, D. On the sample efficiency of abstractions and potential-based reward shaping in reinforcement learning (2025). Available from: https://arxiv.org/abs/2404.07826, arXiv:2404.07826.

[6] Cipollone, R., Favorito, M., Maiorana, F., De Giacomo, G., Iocchi, L., and Patrizi, F. Exploiting robot abstractions in episodic rl via reward shaping and heuristics. *Robotics and Autonomous Systems*, **193** (2025), 105116. doi:10.1016/j.robot.2025.105116.

[7] De Giacomo, G. and Vardi, M. Linear temporal logic and linear dynamic logic on finite traces. *IJCAI International Joint Conference on Artificial Intelligence*, (2013), 854.

[8] Dietterich, T. G. Hierarchical reinforcement learning with the maxq value function decomposition (1999). Available from: https://arxiv.org/abs/cs/9905014, arXiv:cs/9905014.

[9] Mnih, V., et al. Human-level control through deep reinforcement learning. *Nature*, **518** (2015), 529. doi:10.1038/nature14236.

[10] Ng, A. Y., Harada, D., and Russell, S. J. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning*, ICML '99, p.

278–287. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA (1999). ISBN 1558606122.

[11] Parr, R. and Russell, S. Reinforcement learning with hierarchies of machines. In *Advances in Neural Information Processing Systems* (edited by M. Jordan, M. Kearns, and S. Solla), vol. 10. MIT Press (1997). Available from: https://proceedings.neurips.cc/paper_files/paper/1997/file/5ca3e9b122f61f8f06494c97b1afccf3-Paper.pdf.

[12] Pnueli, A. The temporal logic of programs. In *18th Annual Symposium on Foundations of Computer Science (sfcs 1977)*, pp. 46–57 (1977). doi:10.1109/SFCS.1977.32.

[13] Sutton, R. and Precup, D. Between mdps and semi-mdps: Learning, planning, and representing knowledge at multiple temporal scales. (1998).

[14] Sutton, R. S. and Barto, A. G. *Reinforcement Learning: An Introduction*. The MIT Press, second edn. (2018).

[15] van Hasselt, H., Guez, A., and Silver, D. Deep reinforcement learning with double q-learning. In *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, pp. 2094–2100 (2016).

[16] Watkins, C. J. C. H. and Dayan, P. Q-learning. *Machine Learning*, **8** (1992), 279. doi:10.1007/BF00992698.